

Examen Práctico: Mantenimiento de Software

*Evolución, instrumentación de pruebas, logs y aseguramiento de la mantenibilidad en
arquitectura CRUD*

Construcción de Software 2026 · GR2SW

El Desafío del Examen

Proyecto: EPN Event Manager

Punto de Partida

CRUD inicial en NestJS con deuda técnica identificada:

- DELETE sin persistencia (save() ausente)
- Fechas en formato local variable
- getStats() con conteo incompleto
- Sin validación de entrada en DTOs

Objetivo del Examen

Implementar los 4 tipos de mantenimiento normados:

- ✓ Correctivo — bugs & logging
- ✓ Adaptativo — entorno institucional
- ✓ Perfectivo — pruebas & documentación
- ✓ Preventivo — programación defensiva

Diagnóstico: Deuda Técnica Identificada

Análisis crítico del código base original — EPN Event Manager

BUG CRÍTICO

DELETE sin persistencia

events.service.ts: bloque DELETE construía el objeto pero nunca llamaba a save(). Los eventos se perdían silenciosamente.

INCONSISTENCIA

Formato de fechas variable

toLocaleString() producía formatos distintos según el entorno del servidor, rompiendo parseo downstream.

LÓGICA ROTA

getStats() incompleto

query_events no era incluido en el total de estadísticas, devolviendo métricas incorrectas al consumidor.

SEGURIDAD

DTOs sin validación

CreateEventDto aceptaba cualquier payload sin validar tipos, rangos ni campos requeridos — vector de inyección.

1. Mantenimiento **Correctivo**

CORRECCIÓN DE BUGS

Problema identificado:

```
// events.service.ts - bloque DELETE  
const ev = this.deleteRepo.create(dto);  
// ← save() AUSENTE: datos perdidos
```

Solución aplicada:

```
const ev = this.deleteRepo.create(dto);  
await this.deleteRepo.save(ev); // ✓ FIX
```

LOGS ESTRUCTURADOS (WINSTON)

- ✓ Logger via nest-winston con timestamps ISO 8601
- ✓ Niveles: INFO / WARN / ERROR en cada operación CRUD
- ✓ Trazabilidad en consola y hoja de archivo
- ✓ Formato estructurado: {timestamp, level, context, message}

```
[2026-05-02T14:23:11.482Z] INFO [SuscripcionesService]  
CREATE OK id=a1b2  
[2026-05-02T14:23:15.001Z] WARN [SuscripcionesService]  
UPDATE id=xyz not found  
[2026-05-02T14:23:18.773Z] ERROR [SuscripcionesService]  
DELETE failed: DB error
```

2. Mantenimiento Adaptativo

Adaptación al entorno institucional de la Facultad de Ingeniería de Sistemas — EPN

Seguridad por API-Key (X-FIS-EPN-KEY)

- Guard global ApiKeyGuard implementado en NestJS
- Valida cabecera X-FIS-EPN-KEY en cada request
- Responde 401 si la clave es inválida o ausente
- Registrado en AppModule como guard global

Configuración Externa (.env)

- ConfigModule integrado en AppModule
- Variables: PORT, DB_PATH, FIS_EPN_KEY
- .env.example creado para documentar variables esperadas
- Sin secrets hardcodeados en el código fuente

Estandarización de Fechas

- toLocaleString() → new Date().toISOString()
- Formato UTC estándar ISO 8601 en toda persistencia
- Compatibilidad garantizada entre entornos del servidor

3. Mantenimiento Perfectivo

SUITE DE PRUEBAS UNITARIAS (JEST)

- ✓ `create()` retorna suscripción con id único
- ✓ `findOne()` lanza `NotFoundException` si id no existe
- ✓ `update()` aplica cambios parciales correctamente
- ✓ `remove()` lanza `NotFoundException` si no existe
- ✓ Precio negativo rechazado por DTO validation
- ✓ Nombre vacío rechazado por DTO validation

DOCUMENTACIÓN ESTANDARIZADA

Colección Postman

4 endpoints con variables baseUrl, apiKey, suscripcionId y ejemplos de request/response

README completo

Instrucciones de ejecución, guía de demo, checklist de examen y clasificación de mantenimientos

Archivo demo.http

Llamadas curl listas para demostración en vivo incluyendo caso de precio inválido (400)

.env.example

Variables PORT, DB_PATH, FIS_EPN_KEY documentadas para portabilidad institucional

4. Mantenimiento Preventivo

Programación defensiva para blindar el sistema antes de que los errores lleguen a producción

Validación con class-validator

DTOs con decoradores `@IsString()`, `@IsNumber()`, `@Min(0)`, `@IsDateString()`
+ `ValidationPipe` global en `main.ts` con `transform: true` y `whitelist: true`

Filtro Global de Excepciones

`AllExceptionsFilter` captura cualquier excepción no controlada
Devuelve respuesta estructurada: `{statusCode, message, timestamp, path}`
Evita que el servidor caiga ante payloads malformados

Try-Catch por Operación

Cada método del Service envuelto en try-catch
Errores de DB no llegan al usuario final como stack traces
Logger registra el error con contexto antes de relanzar

NotFoundException explícito

`findOne()` y `remove()` verifican existencia antes de operar
Lanza `NotFoundException` con mensaje descriptivo (HTTP 404)
Evita errores de referencia nula en runtime

Módulo suscripciones — Estructura Final

ESTRUCTURA DE ARCHIVOS

```
e pn-event-manager/  
  src/  
    main.ts           ← Guards + ValidationPipe  
    app.module.ts      ← ConfigModule + ApiKeyGuard  
    modules/suscripciones/  
      suscripciones.controller.ts  
      suscripciones.service.ts  
      dto/ (create + update)  
      suscripciones.service.spec.ts  
    common/  
      guards/api-key.guard.ts  
      filters/all-exceptions.filter.ts  
  .env.example  
  postman/suscripciones.postman_collection.json
```

ENDPOINTS IMPLEMENTADOS

POST /suscripciones

Crear suscripción con validación · HTTP 201

GET /suscripciones/:id

Obtener por ID (404 si no existe) · HTTP 200

PUT /suscripciones/:id

Actualizar campos (validación aplicada) · HTTP 200

DELETE /suscripciones/:id

Eliminar (404 si no existe) · HTTP 200

Todas las rutas requieren header: X-FIS-EPN-KEY

Rúbrica — Cumplimiento del Examen

1. El Diagnóstico

25%



Análisis crítico de los 4 bugs de deuda técnica identificados en el código base original

2. Intervención Técnica

35%



Refactorización completa: suite Jest (6 pruebas), Winston logger, corrección DELETE, validación DTO

3. Clasificación y Teoría

20%



Justificación formal de cada cambio mapeado a taxonomía (Correctivo/Adaptativo/Perfectivo/Preventivo)

4. Prueba de Vida

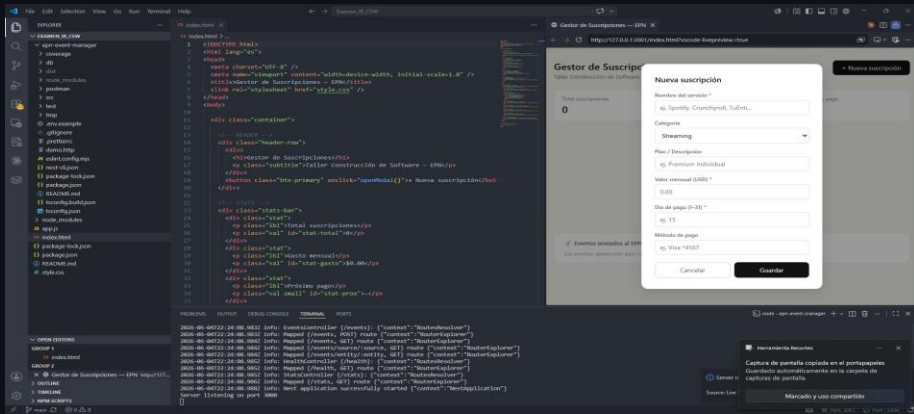
20%



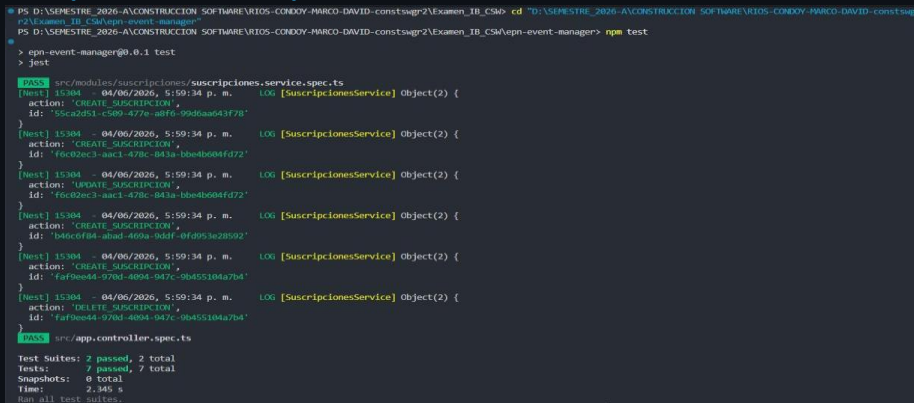
CRUD estabilizado, responde a peticiones inválidas con 400/404 y logs visibles en consola

5. Evidencia de la Prueba de Vida

✓ Frontend + Backend levantados



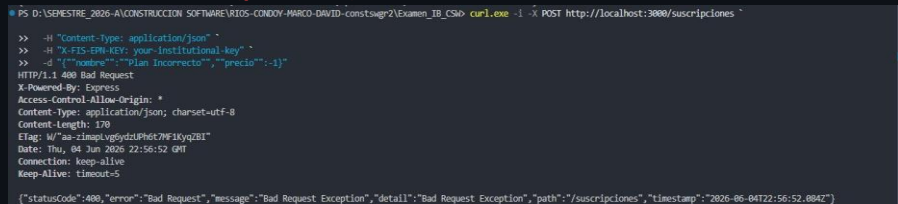
✓ npm test — 7 tests passed, 2 suites



✓ POST válido → HTTP 201 Created



✓ POST inválido (precio -1) → HTTP 400



Resumen de Evidencias

- ✓ Servidor NestJS activo en puerto 3000
- ✓ CRUD frontend funcional (localStorage)
- ✓ POST válido → HTTP 201 + id único generado
- ✓ Precio negativo → HTTP 400 (validación activa)
- ✓ 7 pruebas Jest — 2 suites PASS
- ✓ Logs con timestamps ISO 8601 visibles